Silicon-Stack Final Report

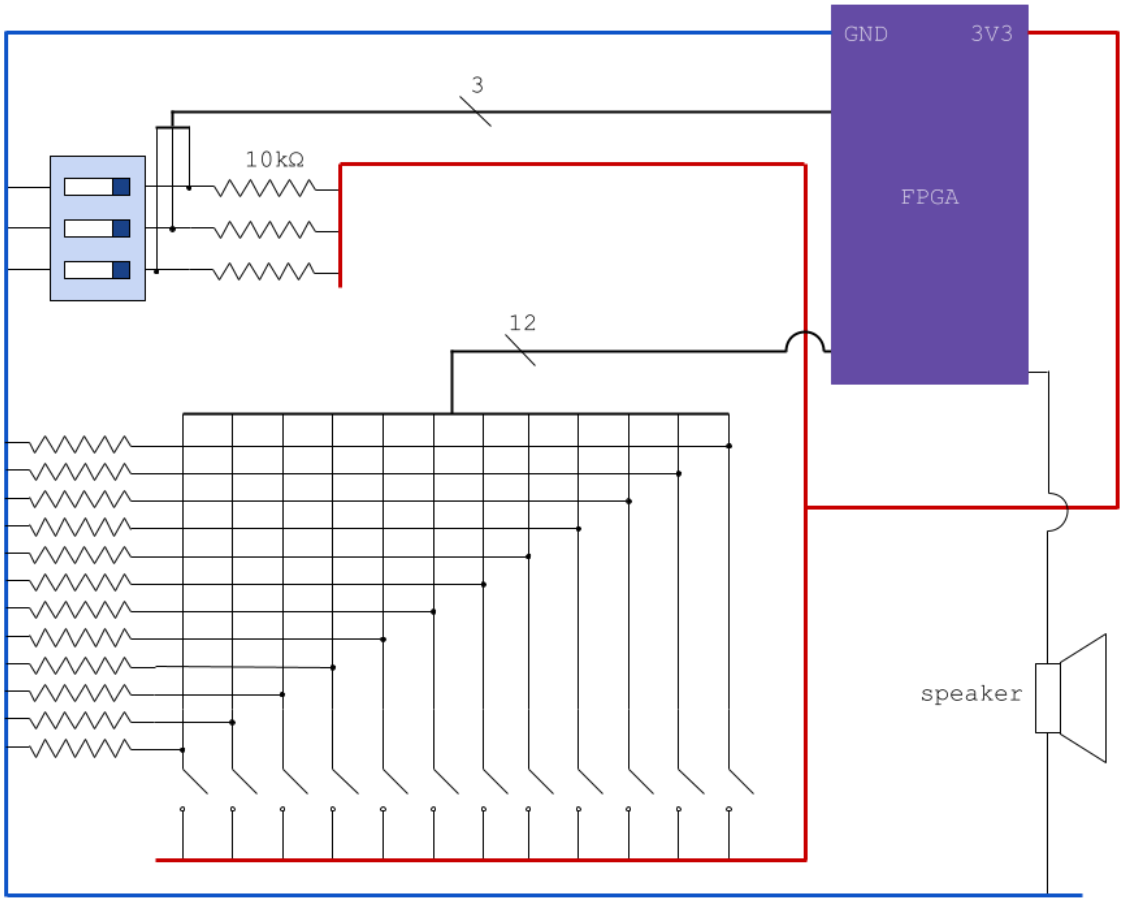
12/18/2026

Mason Sibley, Charlotte Yamamoto, Ariadne Colina-Valeri, Selin Tuncay

Overview Pre-Demo Report:

The goal of our project is to create all the functionalities of a keyboard. While the other part of our group works on the MIDI we worked on getting the keyboard to sound like a keyboard. A normal keyboard is able to have 12 correct notes per octave, several fully functional octaves depending on the keyboard, and playable cords when playing difficult music. These were the features we were most interested in creating.

Technical Descriptions:



Caption for Picture 1: Full Board

NOTE FREQUENCY CHART | HEROIC AUDIO

	Octave 0	Octave 1	Octave 2	Octave 3	Octave 4	Octave 5	Octave 6	Octave 7	Octave 8	Octave 9	Octave 10
C	16.35	32.70	65.41	130.81	261.63	523.25	1046.50	2093.00	4186.01	8372.02	16744.04
C#	17.32	34.65	69.30	138.59	277.18	554.37	1108.73	2217.46	4434.92	8869.84	17739.69
D	18.35	36.71	73.42	146.83	293.66	587.33	1174.66	2349.32	4698.64	9397.27	18794.55
D#	19.45	38.89	77.78	155.56	311.13	622.25	1244.51	2489.02	4978.03	9956.06	19912.13
E	20.60	41.20	82.41	164.81	329.63	659.26	1318.51	2637.02	5274.04	10548.08	
F	21.83	43.65	87.31	174.61	349.23	698.46	1396.91	2793.83	5587.65	11175.30	
F#	23.12	46.25	92.50	185.00	369.99	739.99	1479.98	2959.96	5919.91	11839.82	
G	24.50	49.00	98.00	196.00	392.00	783.99	1567.98	3135.96	6271.93	12543.86	
G#	25.96	51.91	103.83	207.65	415.30	830.61	1661.22	3322.44	6644.88	13289.75	
A	27.50	55.00	110.00	220.00	440.00	880.00	1760.00	3520.00	7040.00	14080.00	
A#	29.14	58.27	116.54	233.08	466.16	932.33	1864.66	3729.31	7458.62	14917.24	
B	30.87	61.74	123.47	246.94	493.88	987.77	1975.53	3951.07	7902.13	15804.26	

Caption for Picture 2: Frequency chart used to analyze notes per octave

Results:

Goal:
We have a full scale of 12 notes which plays “Do-Re-Mi” with sharp and flats (the black keys). We achieved 8 full octaves from 16Hz to around 8,000Hz. Have chords that blend the notes together letting us play 6 notes at once at the lowest octave all notes together at the highest octave. Having the sound come out directly through the speaker was the best option giving us the best sound.

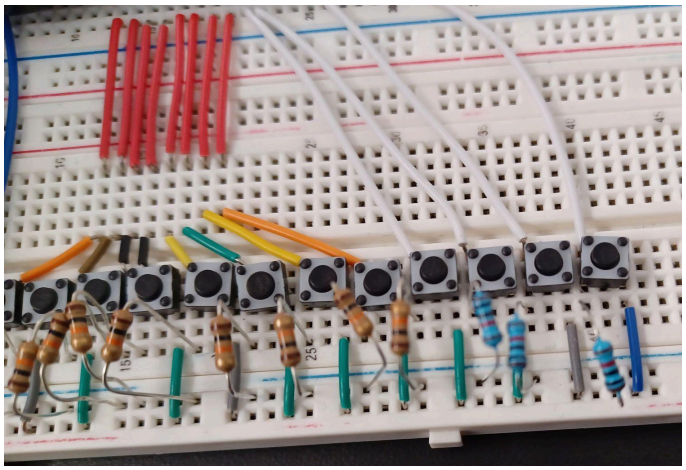
Known Bugs:
The Volume Trade off: If we want the chords to sound super clear with at least 6 notes at a time the volume must remain at a medium/loud. If we try to make it “extra loud” in verilog by expanding the bit count the chords start to sound static, if you want more than 6 very clear simultaneous notes the bit count would have to increase, but the volume decreases in turn.

Future Improvements:
The first improvement is to add a new Amp so there is no trade off. The bit count on the PWM would be put to its highest, and it won’t matter because the Amp would compensate. This way the Amp allows for a good amount of volume while also getting all playing notes at each octave.

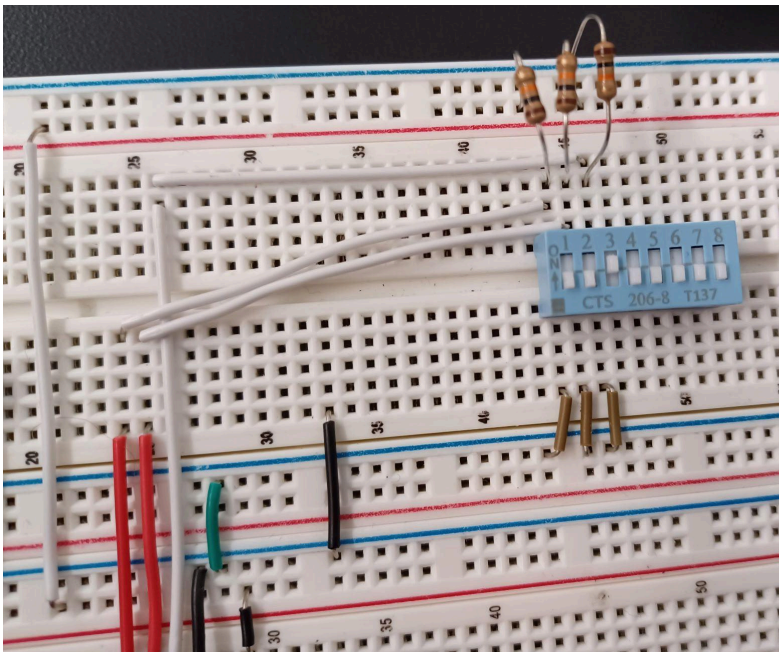
The second improvement is adding a forte(Louder),piano(Quieter), and sustainer pedal such as piano pedals. The plan is to do this within the MIDI when everything is connected. The velocity of the pressed key would determine the volume of the note. As for the sustain pedal the key would keep going until the note is not pressed.

The third and final improvement would be to add “voices” as a real keyboard has. So that the sound played could be the correct note as a duck, flute, guitar, or a trumpet. At the moment we use square waves, instead, we would change the logic inside the FPGA. We would tell the FPGA to 'fade' the signal up and down slowly during each period instead of jumping. So that if a combination is put into the keyboard, for example, pressing a switch twice and holding could be flute and three times could be guitar.

ScreenShots:

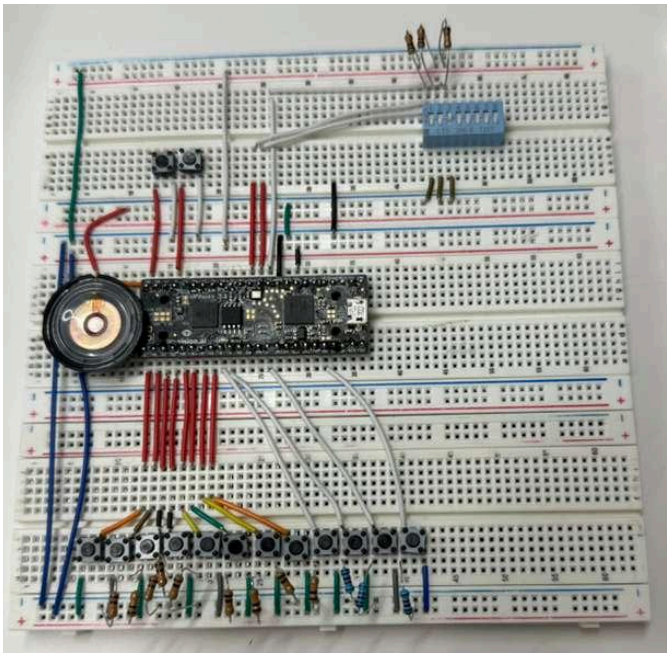


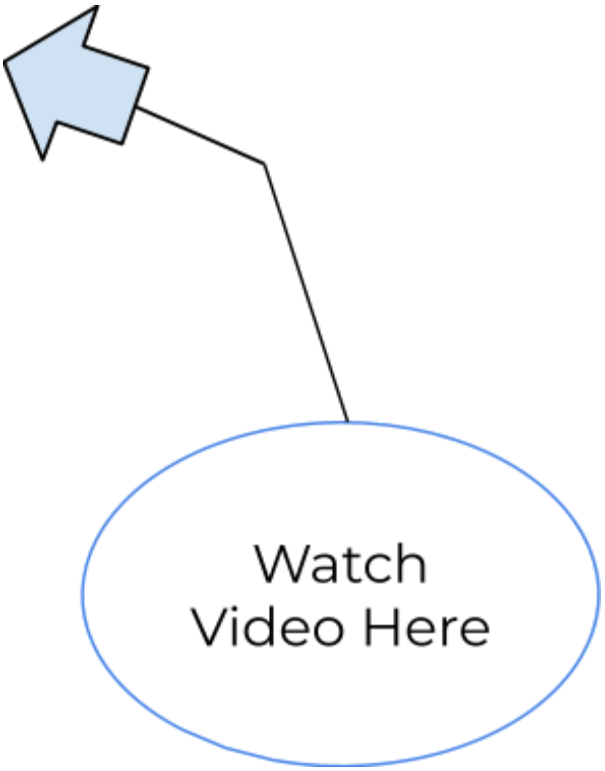
Caption for Picture 3: 12-Notes and Chords



Caption for Picture 4: Octaves

Watch Video Here





Caption for Picture 5: Full Board and *Video of 12-Notes, Working Octaves, and Chords*

Reflection:

12 notes:

Since the MIDI had not been processed yet in order to be connected we decided to test out the notes with buttons that connect directly to the FPGA. We also used lights in order to see if they were turning on correctly when at times no sound came out.

Problem: the speaker simply refused to make noise

We tried to test the speaker with the amplifier chip and an independently generated sine wave, but when it was connected, the chip was hot and potentially fried, in addition to not making sounds. Despite the efforts of the present group members and TAs, the speaker did not produce audio output that day.

The next time we went into the lab, we found that our amplifier chip was missing, although the speaker was still there. We resolved to simply use the speaker without the amplifier chip. Plugging the speaker inputs directly into the signal allowed us to produce audio output.

Problem: could not figure out how the clock-periods had been calculated for the octave that had been programmed

Attempting to hand-calculate the clock-periods of the fourth octave using the information in the code resulted in values that differed greatly from the clock-periods that were present in the code. Through trial and error, we discovered that these calculated periods were off by a factor of 2. Using a tuner, we determined that the notes being produced by the speaker were an octave lower than the code stated they would be.

Using this information, we deduced that the oscillator was actually operating at double the frequency that the code had suggested. Instead of the frequency being octave 4 it was actually octave 3 which was a great difference. This information made it possible to create a scale of notes excluding the black/half notes without having issues on the active octave. By going into the code and extending the notes to 12 notes we were able to create a full octave.

Octave:

Problems with Octaves: Instead of manually calculating the period of each of the 128 potential notes, we decided to utilize the fact that the frequency of a note (for example, A3 at 220 Hz) doubles when the octave is increased (A4 is 440 Hz). Division in any computer is a complex process, taking up a lot of space in the FPGA. The FPGA didn't have enough LUTs to implement this division process.

Manual Product Description: Q25 - Quickstart Guide - RevA

After achieving a 12 note octave, we moved on to attempting to get different octaves. We knew that the keyboard started at octave 4 and had the ability to move two up and two down after finding the manual product description.

At the beginning we had the idea of optimizing the operations done by the computer so that we didn't have to hand-calculate as many clock-periods. We took the frequency of octave 4 and used it as a base, and had a module that went off the base frequency to divide by two and multiply by two when trying to achieve different octave frequencies. We had the formula of the frequency of the clock divided by the frequency of the note running through calculations of each of the octave frequencies to create an output of a selected octave note. We thought that this would create an easier time for us as the creator until we came across memory overloading. We tried different things but in the end we realized that the memory was over 250% because of the division function that was being used to calculate the octave notes. We realized that division in computers is a lot of operations that take a few clock cycles to process and this is where the issue originated.

How it was fixed: Created a logical shift, 2x to the left or the right, instead dividing by 2.

In the end, we decided that it would be easier to hand-calculate the clock-periods for one octave of notes (dividing the clock frequency by the note frequency), rather than making the FPGA do it every time.

Instead of manually calculating the clock-period (number of clock cycles in a period of the note) for each note of each of the 6 octaves, we decided to utilize the fact that the frequency of a note (for example, A3 at 220 Hz) doubles when the octave is increased (A4 is 440 Hz), meaning the period is cut in half. Initially, we implemented this using division, since the period is divided in half for every octave. However, we soon realized that a quicker way to do that would be to use a logical shift, since all of the divisions and multiplications were by powers of 2.

For maximum accuracy and range, we hand-calculated the largest clock-periods, which were for the first octave. We relied on logical right-shifts to divide by powers of 2 for increasing octaves.

Problem: testing octave range

In order to test the range of octaves, we attempted to mimic the “octave up/down” buttons on the MIDI keyboard with push-button switches. When this was implemented, the octave of the audio output varied irregularly. Trying to make one push of a button increase or decrease the octave number by one did not work, resulting in semi-random octave numbers when the switches were pushed. We checked the octave number by outputting it in binary using the LEDs. We ended up deciding that since the push-button switches would not be part of the final product, there must be a better way to change the octave.

To do this, we connected a DIP switch that represents the number of the octave in binary with 3 bits to span the 6 octaves that the MIDI keyboard could produce. This method worked much better, and the octave-number LEDs were removed.

Chords & Low Notes:

Problem Low notes: The lowest note that could play was F#2 so the octave was not completed

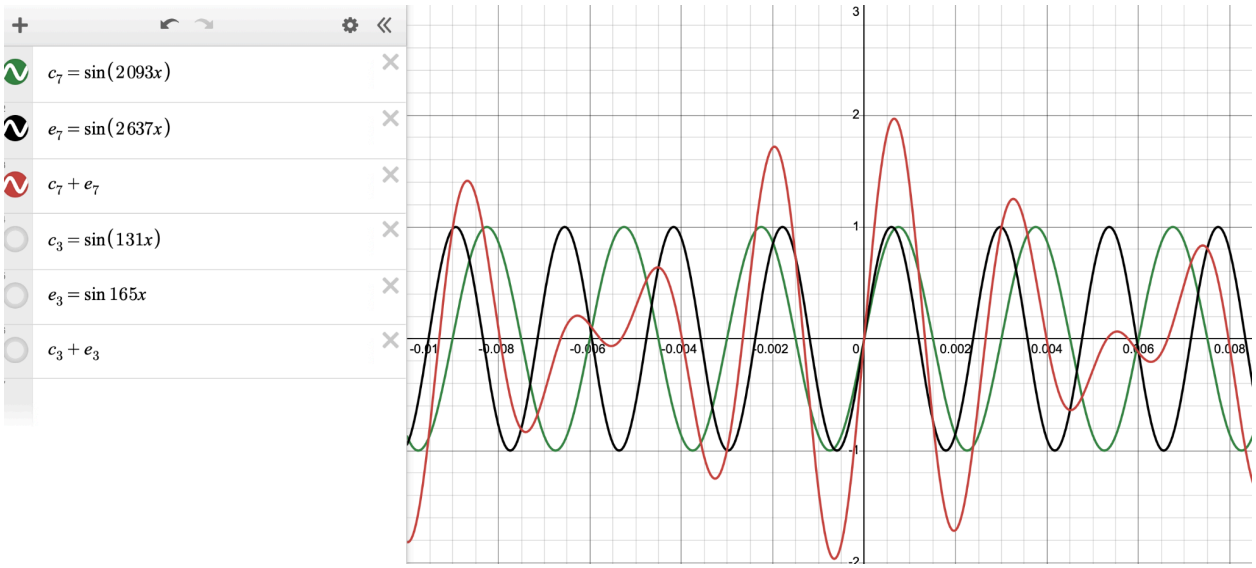
How it was fixed: Increasing the number of bits of the period count

After the octaves were created correctly we wanted the sound of the buttons to take on the same attributes of an actual keyboard. The range that we had at the time went from F#2 to C7. In order to fix it, we increased the bit count on the bus holding the clock-period count so that the speaker could produce notes lower than about 92Hz. That is how we achieved full 8 octaves. When that was done we decided to try and achieve chords, which was a much more difficult process as we did not know how that could be put in SystemVerilog.

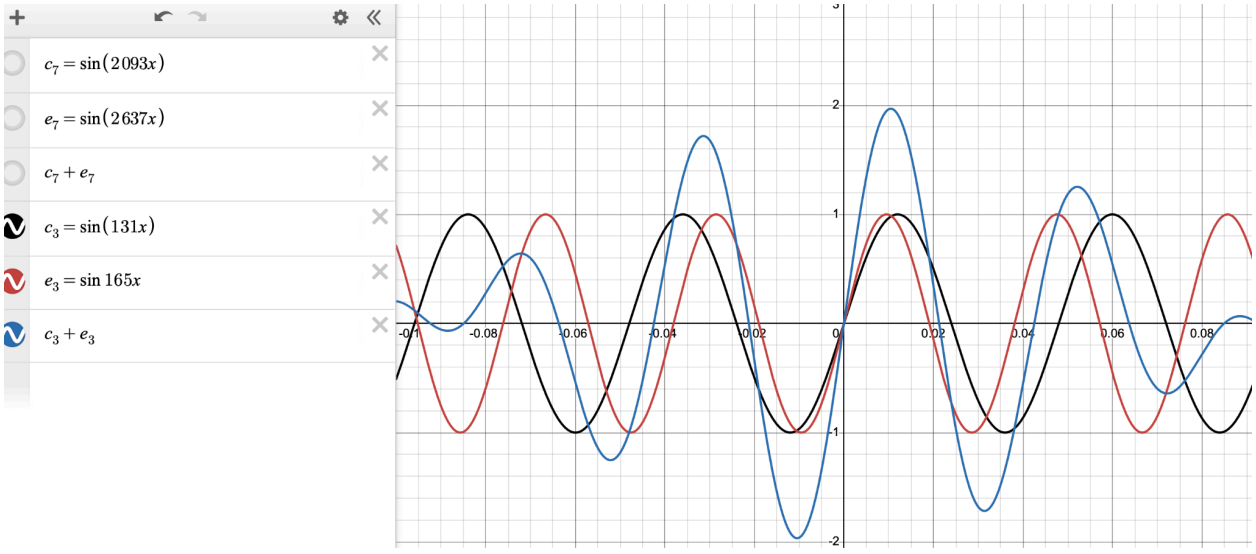
Problem with Chords: When pressing more than one note at the same time it created discord and static noise.

With the working notes at the octaves functioning correctly, we could produce chords of 2 notes when in a high octave. However, when we added a third note or dropped to an octave lower than 6, even 2 notes playing simultaneously created a static discord noise. We ended up analyzing wave forms to see how the frequencies looked on a graph and why the joint frequencies of high notes were working when the low were not. On Desmos we plotted the frequencies C and E for octave 7 and the same for octave 3, then we realized that the problem came with the addition of the two sine waves. The ranges of values for which the sine waves of the individual notes were above zero did not correspond to the ranges of values for which the summed sine wave was above zero.

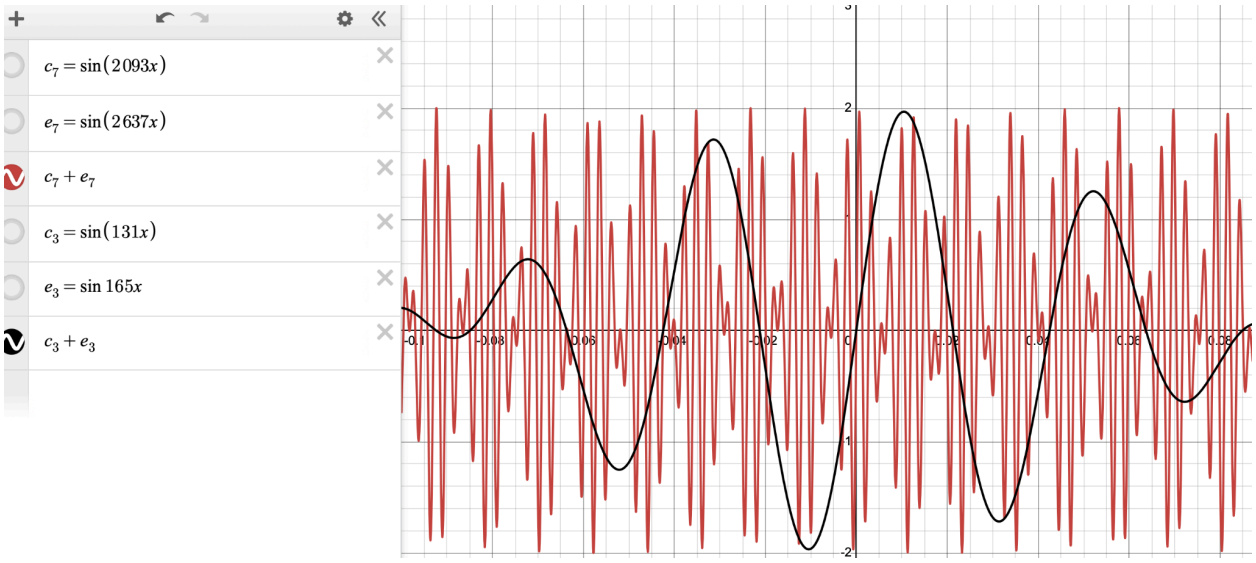
The overall amplitude remained the same but the amount of waves created in the same time was higher for octave 7 than for octave 3. We came to the conclusion that the system was taking in the start of the first wave note and skipping to the end of the second wave note. A chord is the addition of the notes together. This explained that the output we were producing was trying to process the two individual notes at the same time, ignoring the constructive and destructive interference of the addition of the two sine waves.



Caption for Picture 6: Octave 7 on C and E at the same time



Caption for Picture 7: Octave 3 on C and E at the same time



Caption for Picture 8: Two notes played at different Octaves

How it was fixed: Used the PWM method

We tried different things until we discovered The Pulse-Width Modulation (PWM) mixing technique which solved the width/period cycle problem because it controls the period cycle based on the summed amplitude. In this sense, we got the notes to join by combining. The result of this method was fully working chords.

Sound & Op-Amp:

Problem with Sound Volume: The chords worked but the sound was really low

In the end, this created amazing chords but dropped the volume quite a bit. For this reason, we tried to connect it to an op-amp given to us in our EE20 class, as our amplifier chip had gone missing. However, this disrupted the PWM, getting rid of the chords, although the volume was back up. We figured that this was likely due to the cutoff from the op-amp, and that if the gain were to be lowered, we might be able to increase the volume and preserve the chords.

How it was fixed: We lowered the bit count on the chords so that the sound could increase.

We tried to reduce the gain of the op-amp as much as we could, but despite our best efforts and many trials, the complex sound never returned. We then decided to modify the SystemVerilog code, rather than the hardware.

After some testing, we found that the more bits were allowed on the PWM bus, the more complex the audio output became, while at the same time the volume decreased. We came to the conclusion that a bus of 3 bits was the perfect midpoint of volume and complexity. We ended up with relatively med/loud volume and 5-6 note chords on the lowest octave while the highest octave could produce complex sounds even with all of the notes playing.

Overview Post Demo Report:

Our project was partially complete to the degree we wanted before the demo on Monday. After the demo we decided to re-organize most of the project to try and achieve our goals. One of the major things we were not able to accomplish was the MIDI communication via fpga. This was the first challenge we needed to tackle. After the MIDI communication was figured out, the world opened up as that was the last major hurdle. In terms of what changed from the first design to the second, we scrapped the way we classified notes in the code, added outputs that would help with troubleshooting, and implemented an easy to follow MIDI UART module.

Technical Description:

The MIDI part is responsible for turning the keyboard actions such as pressing and releasing into a clean, synchronous set of signals the FPGA's audio/tone pipeline can use for selecting notes, octave positions and activating notes. In the early development, our audio chain was validated without MIDI by using push-buttons and LEDs directly on the FPGA to confirm note selection and debugging when audio output was inconsistent. Once the sound output was stable, we replaced the manual inputs with decoded MIDI notes.

We did not end up getting the MIDI processor to work inside the FPGA before the demo for a couple key reasons that we found out later. MIDI itself is a communications protocol that is based somewhat in UART. We were able to manipulate this into giving us UART signals at 31250 bits per second. The way MIDI data is organized is split into 1 start bit, 8 data bits, and 1 stop bit. This in itself creates one MIDI word. MIDI communications has 3 words in a message; key on/off, what key, and velocity that the key was pressed.

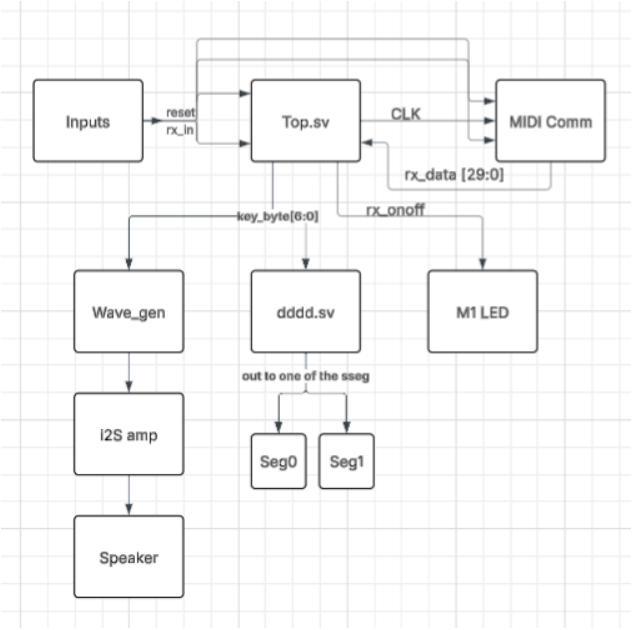
For the actual MIDI communication we ended up adopting a UART bit reader and redesigning it for this specific use case. Instead of having one UART word come through, we would read 3 at a time if a flag came up. This resulted in a reading and storage of the data stored in the format below.



Caption for Picture 9: Visual of MIDI message format

System verilog modules and code design:

At a high level our new pipeline looks like this:



Caption for Picture 10: Visual of final project layout

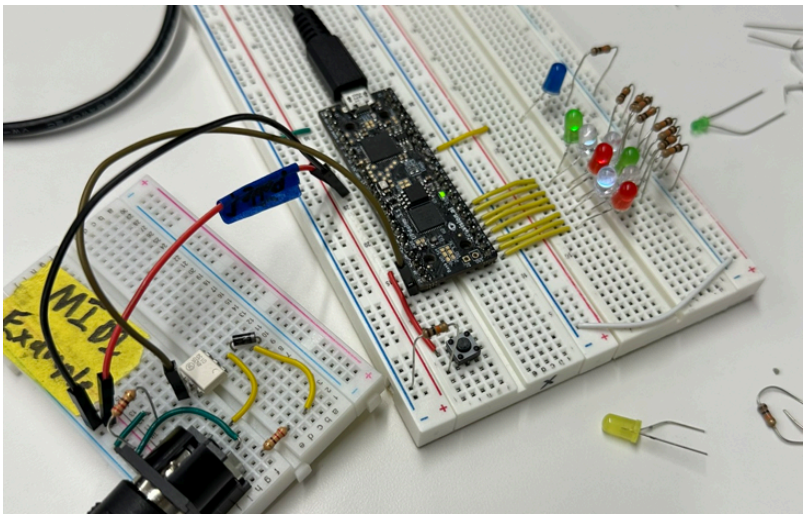
MIDI keyboard -> MIDI byte stream -> UART -> wavegeneration, 7seg out, and one LED.

MIDI Communication Modules:

UART reader:

We retrofitted a UART reader into our project to sample 30 bits in continuation if the right conditions were met. Our finite state machine had 4 states: idle, start, data, and stop. The first state was the waiting state, IE waiting for the rx_in to go low to signal there's a message coming. Next is our start state which is only there so we can essentially start the count and shift our timing by half a MIDI pulse, leaving us sampling in the middle of the clock edge. Next is our data state where we wait for one UART bit to come through and then sample, and repeat until we have hit 30 bits. Last is stop, which sends us back to idle.

For the MIDI protocol, it caused a lot of confusion but simpler was better in the end. From the 30 bit message we got, we could extract what we needed from it: is a key pressed, what key, and how hard that key is pressed. Once we thought we had something working, we had a very hard time figuring out if it WAS actually working. This is when we decided to put In several LED's as an output to test if it was actually getting each message correctly.



Caption for Picture 11: Picture of 8 bit LED prototype

The LED's were set to function as an 8 bit number that would output whichever word from MIDI we wanted. This validated our results.

MIDI LUT:

Because we are sampling sine waves for sound, we needed to base that off of the clock we are running at, 24MHZ. This comes out to 768 clock cycles per MIDI clock cycle. That would be our ground sampling rate. Furthermore, because we were sampling sine waves, we weren't able to use 1's and 0's but rather we needed something that could fit into I2S.

We started by thinking about the phase increase per time that we would get each measure for a certain frequency. At 440 hertz we would increase the phase 654. Doing the calculations for all 128 notes starting at C-1 (8.18hz) all the way up to G-9 (~12500hz).

This LUT allowed us to grab pre-calculated values rather than creating logic that could slow our clock even further, and give us less clutter in our program. A particular advantage to this is that our program does see octaves anywhere. To this design, the keyboard is laid out 128 keys, side by side. There is no mention of octave as that is just something we do to section us frequency. This creates a streamlined waveform generation.

Wave Generator:

The output of all of this was in the wave generator. This wave generator was made so that It could have 2 notes playing at the same time and output in a way i2S could understand. We had the clock, each respective note number, the note on and off variable, and the velocity of the first note (we did not have time to modify for the second note).

Essentially all this module did was for each note we would sample the sine wave: At each clock cycle, find the previous value, and add the phase increment to the 16 bit number. This would be our sample we could output into the I2S one we apply our velocity to get volume.

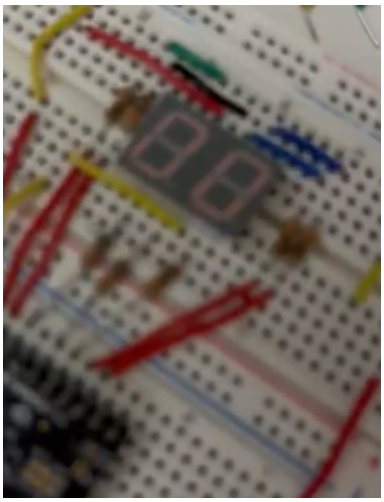
When two notes were on at the same time, the waveforms would be summed together. This single output would be sent into the amp and then into the speaker. As for the I2S protocol, we translated our clock into the bit clock that I2S requires, and the audio sample is shifted out, MSB first, in sync with our clock.

RAM Use and Modules:

With the use of MIDI protocol, we found that we needed to save if a key was pressed or not. This is where our ram module came into play. First we retrofitted the ram lab module into our top.sv and then added two notes to be on at the same time. Each ram entry corresponds to a note number bit, either on or off. The note number is used as the write address and the on/off state is what is written into memory. This separates UART activity from the system allowing us to keep track of multiple notes and make a polyphone. We used the rampd.sv module given to us for lab 7.

Troubleshooting Design and Modules:

In this final design, a 7 segment and LED was incorporated. This was because there is no easy way to validate your program through terminal or output other than what you actually wire. So we decided to wire a 7 segment display that has alternating outputs, like we learned in Lab 5.



Caption for Picture 12: 7 segment showing first key pressed

Moreover we incorporated a singular LED to show whenever a key was pressed, representing the old 8 bit number. We were either looking for 1001000, or 1000000 which represented h90 and h80. Because this was simple enough we decided to give the output a binary value and give it to a blue LED.

The modules for this are the dddd.sv and decoder.sv. We are not going to explain them here as they are in other lab reports from all of us and the code will be on github.

Results:

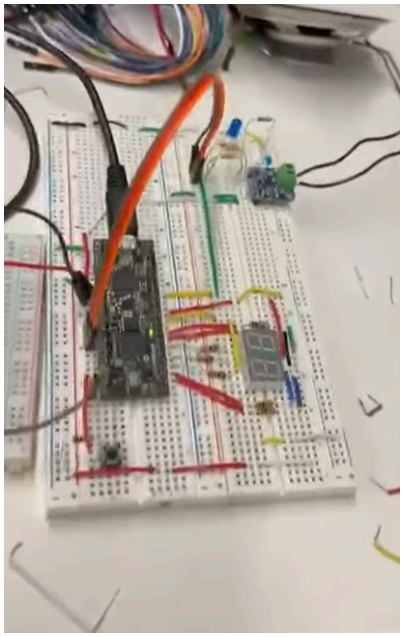
For our design, we started off simple. For proof of concept we wanted to see if we could make an LED light up when a certain key is pressed. This was challenging to say the least. For this to happen, we needed a couple of things. We needed to recognize that the first key is being pressed, which was pretty simple (look for byte1 == h90). The second and third keys were harder. We first tried to recognize them by shifting the bytes in a shift register each clock cycle. When we see h90, a flag goes up and we record the next 2 bytes for the clock cycle. This ended up working as we got the first LED to recognize the key was pressed and a second to recognize that a certain key was pressed.

This method came at a cost, however. When we went to press more than one key at a time, regardless of whether you were still pressing the first key, if you let go of the second, the program would register h80 and kill all sound.

Post demo when we finalized our design, we were able to get much better results. Overall we still had the problem of killing all sound when one key was turned off, however, we were able to make two notes sound at the same time through the i2S speaker USING MIDI communication. Hearing the output from the speaker was enough to make a grown man cry. Though it was not *exactly* what we wanted, it was enough to declare victory against the *evil MIDI keyboard*.

A video of our keyboard working and being tested will be uploaded to the github





Watch Video 1 Here:

Watch Video 2 Here:

Reflection*:*

This was a really interesting thing to try and figure out. We spent way too much time trying to brute force through this instead of stepping back and trying to figure this out from the ground up. For example, trying to get the LED to light up took way too long and when we figured out how to do it, we were already a little behind, and the next day we worked until the demo presentation. From a different perspective, this experience changed the way we thought about debugging and design. Instead of assuming the whole system should work at once, we started to see the value of building things up in small sections. Getting a single LED to respond correctly sounds successful, but it showed how easy it is to lose track of signals and assumptions when everything is wired together at the same time.

All of this process made us more aware of how we used our time. Spending so long on what was supposed to be a simple check meant we had to rush other parts of the project to get something working for the demo. If we were to repeat this project, we would set clearer intermediate check marks and be more disciplined about stepping back when something “simple” is taking too long. Breaking the problem into smaller steps and tests and confirming that each piece with simple indicators like LEDs would have saved us a lot of stress and given us more time to polish the final system.

Taking the time to redesign the circuit from scratch was helpful because it revealed pathways that were much easier than trying to brute force our way down the path we had spent so much time on already. Looking forward, this project helped us realize that the current path isn't always the right path, and it isn’t always worth sticking around to see it through. Rather it is more beneficial to strategically switch paths and learn from your mistakes. The 7 segment display was a great example, adding one allowed great troubleshooting that saved a lot of time. The display took an hour to set up, which originally is why we didn’t add one.

Work Division:

Audio sound output: Charlotte, Ariadne
Notes: Charlotte & Ariadne
Chords: Charlotte & Ariadne
Octaves: Charlotte & Ariadne
Putting together the Circuit: Charlotte & Ariadne
MIDI UART, LUT modules: Mason & Selin
DDDD, RAMDP, decoder, wave_gen modules: Mason